**BTS SERVICES INFORMATIQUES AUX ORGANISATIONS** **SESSION 2026**

**ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle (recto)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

| **DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE** | **N° réalisation : 1** |
|---|---|
| **Nom, prénom :** RAYANE Hakim | **N° candidat :** 02541507253 |
| **Épreuve ponctuelle** ☒ **Contrôle en cours de formation** ☐ | **Date :** |

**Organisation support de la réalisation professionnelle**

Maison des Ligues de Lorraine — M2L

**Intitulé de la réalisation professionnelle**

Développement d'une application web de réservation de terrains de football 5v5 — « Foot5 Extension Web »

**Période de réalisation :** Décembre 2025 – Mars 2026 **Lieu :** FDME Massy
**Modalité :** ☒ **Seul(e)** ☐ **En équipe**

**Compétences travaillées**

☒ Concevoir et développer une solution applicative
☐ Assurer la maintenance corrective ou évolutive d'une solution applicative
☒ Gérer les données

**Conditions de réalisation[1] (ressources fournies, résultats attendus)**

Ressources fournies :

• Ordinateur personnel (Windows 11), VS Code 1.87, Python 3.13, Node.js 20
• Serveur MySQL local (XAMPP) — base de données « foot5 » partagée avec l'application desktop
• Cahier des charges fonctionnel de la M2L établi en début de projet
• Accès Internet pour les dépendances (pip, npm)

Résultats attendus :

• API REST fonctionnelle avec authentification JWT — 20 endpoints (6 auth + 6 terrains + 8 réservations)
• Interface React responsive accessible depuis tout navigateur, sans installation côté client
• Moteur de réservation : détection des conflits de créneaux, calcul automatique du coût (durée × tarif/h), quota configurable de 2 réservations par semaine (7 jours glissants)

• Suite de tests automatisés (pytest) couvrant les endpoints et la logique métier
• Documentation technique complète avec Swagger auto-généré

[1] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

**Description des ressources documentaires, matérielles et logicielles utilisées[2]**

Matérielles :

• PC personnel (Windows 11)
• Serveur XAMPP local (MySQL 8.0)

Logicielles :

• Éditeur : VS Code 1.87 + extensions Python, ESLint, Prettier
• Backend : Python 3.13, FastAPI 0.115, SQLAlchemy 2.0, Pydantic, python-jose, passlib [bcrypt], uvicorn
• Frontend : React 18, TypeScript 5, Tailwind CSS 3, Axios, React Router DOM
• Base de données : MySQL 8.0 (XAMPP) — Versioning : Git / GitHub

Documentaires :

• Documentation officielle FastAPI, React, SQLAlchemy
• Cours BTS SIO SLAM (bases de données, développement web, sécurité)

**Modalités d'accès aux productions[3] et à leur documentation[4]**

Code source :
→ https://github.com/warkozz/projet-ppe-foot5-web

• Documentation PDF :

Doc. technique : PPE-DOCS-PDF/03_Application_Legere_Web.pdf

Dossier complet : PPE-DOCS-PDF/00_DOSSIER_COMPLET_PPE_FOOT5.pdf

(dans le git)

• Api Swagger : http://localhost:8000/docs (Swagger UI)

Accès en lecture libre (répertoire public GitHub)

[2] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

[3] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

[4] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

**ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle**
**(verso, éventuellement pages suivantes)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

**Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

CONTEXTE

La Maison des Ligues de Lorraine (M2L) gère plusieurs terrains de football 5v5 et souffrait d'une gestion entièrement manuelle (agenda papier, appels téléphoniques), source de conflits de créneaux non détectés et d'une charge administrative élevée. L'organisation a souhaité une solution numérique permettant aux joueurs de réserver en ligne de façon autonome, sans passer par un intermédiaire. L'application web constitue la face cliente du système Foot5. Elle partage la même base de données MySQL que l'application desktop d'administration, garantissant une cohérence des données en temps réel entre les deux interfaces.

OBJECTIF
Développement d'une application web full-stack permettant :
— L'inscription et l'authentification sécurisée des joueurs (JWT, bcrypt)
— La consultation des terrains disponibles avec tarifs et capacités
— La réservation d'un créneau en 3 étapes avec vérification des conflits en temps réel
— La gestion de son espace personnel (historique, annulations)

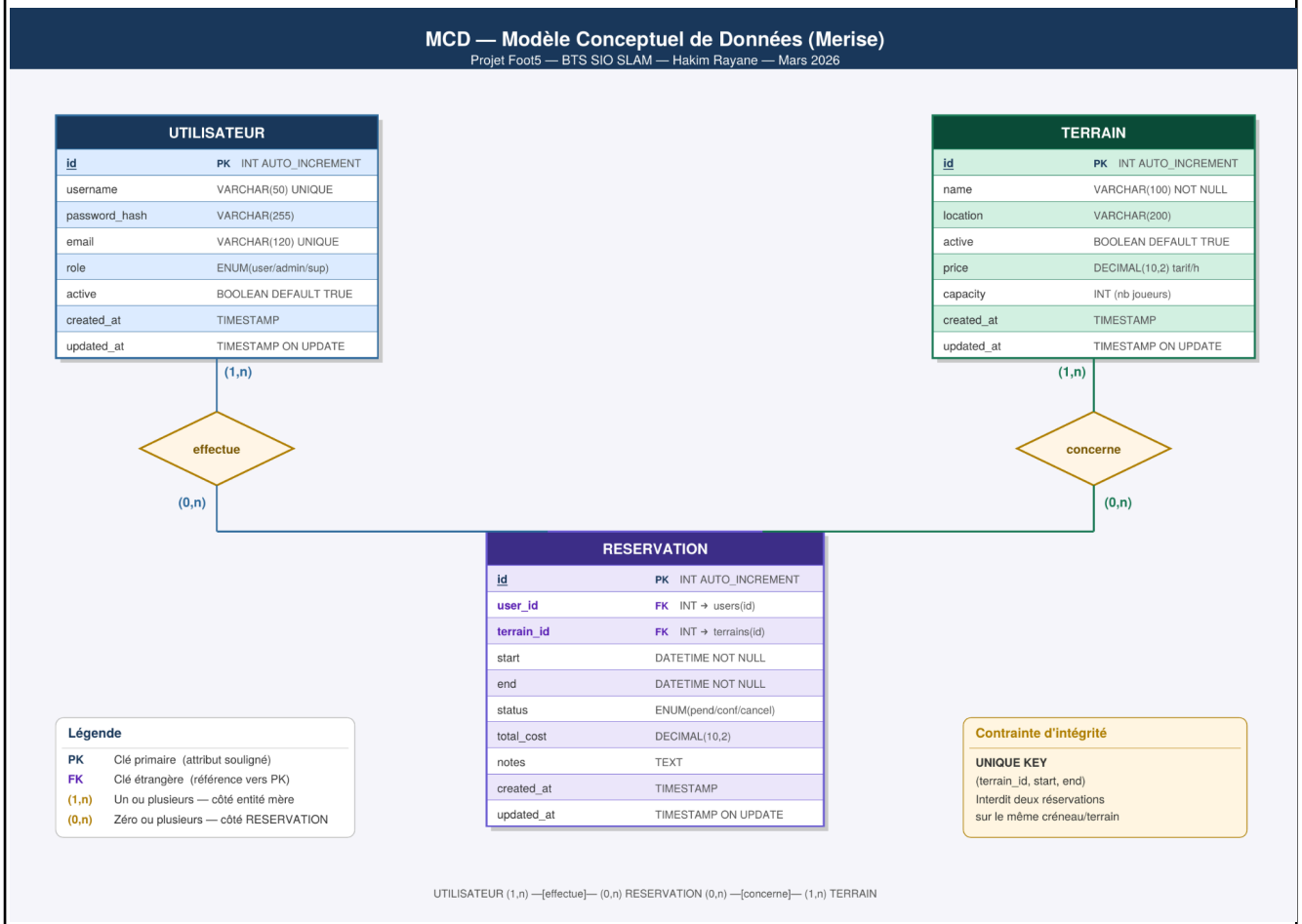
L'application web constitue la face cliente du système Foot5. Elle partage la même base de données MySQL que l'application desktop d'administration, garantissant une cohérence des données en temps réel entre les deux interfaces.

ARCHITECTURE TECHNIQUE
L'application suit le pattern REST API + SPA (Single Page Application) :

Navigateur (React 18 + TypeScript + Tailwind CSS) ↕ HTTP/REST JSON — Axios

Backend API (FastAPI Python — port 8000)
■■■ Authentification : JWT HS256, expiration 30 min
■■■ Validation : schémas Pydantic
■■■ Logique métier : services/ (quota, coûts, conflits)
■■■ ORM : SQLAlchemy 2.0 ↕ MySQL connector (PyMySQL)
Base de données MySQL "foot5" (partagée avec l'app desktop) 3.

Modèle Conceptuel de Données — MCD Merise (base partagée)



RÉALISATIONS PRINCIPALES

1. Module d'authentification : inscription, connexion, refresh token, modification profil

| Endpoint | Description |
|---|---|
| POST /api/auth/register | Inscription avec validation email, hachage bcrypt 12 rounds |
| POST /api/auth/login | Connexion, génération token JWT HS256 (30 min) |
| GET /api/auth/me | Consultation du profil (token requis) |
| PUT /api/auth/me | Modification des informations personnelles |
| PUT /api/auth/me/password | Changement de mot de passe avec vérification de l'ancien |
| POST /api/auth/refresh | Renouvellement automatique du token |

2. Moteur de réservation :

• Détection des conflits : requête SQL sur chevauchement temporel (start < end_new AND end > start_new) sur les réservations pending et confirmed
• Calcul automatique du coût : total_cost = durée_en_heures × terrain.price, calculé côté API à la création
• Quota hebdomadaire : comptage des réservations actives sur les 7 jours glissants précédents — rejet HTTP 429 si ≥ 2 ; valeur configurable via MAX_WEEKLY_RESERVATIONS dans le fichier .env
• Disponibilité : endpoint dédié retournant les créneaux libres pour un terrain et une date donnés

3. Interface React :

| Page | Description |
|---|---|
| Accueil | Présentation du service, boutons Réserver / Se connecter, terrains disponibles |
| Terrains | Cards : nom, localisation, format (5v5), tarif/h, statut (actif/inactif), bouton Réserver |
| Réservation | Parcours 3 étapes : choix terrain → choix créneau (coût estimé en temps réel) → confirmation |
| Mon Espace | Historique des réservations, filtres par statut, tri par date, bouton d'annulation |
| Profil | Modification du compte, changement de mot de passe |

• Composants réutilisables : Badge statut (couleur sémantique), Alert, Navigation

4. Tests automatisés :

| Page | Description |
|---|---|
| Accueil | Présentation du service, boutons Réserver / Se connecter, terrains disponibles |
| Terrains | Cards : nom, localisation, format (5v5), tarif/h, statut (actif/inactif), bouton Réserver |
| Réservation | Parcours 3 étapes : choix terrain → choix créneau (coût estimé en temps réel) → confirmation |
| Mon Espace | Historique des réservations, filtres par statut, tri par date, bouton d'annulation |
| Profil | Modification du compte, changement de mot de passe |

• Composants réutilisables : Badge statut (couleur sémantique), Alert, Navigation
• Gestion d'état global via AuthContext — stockage du token JWT en mémoire
• Charte graphique : vert émeraude #10B981 (Tailwind emerald-500), responsive mobile-first

Contrôle d'accès RBAC :

| Route | Public | User | Admin |
|---|---|---|---|
| GET /api/terrains | ✔ | ✔ | ✔ |
| POST /api/reservations | ✘ | ✔ | ✔ |
| GET /api/reservations | ✘ | Ses resas | Toutes |
| PATCH .../confirm | ✘ | ✘ | ✔ |
| POST/PUT/DELETE /api/terrains | ✘ | ✘ | ✔ |

Tests automatisés — pytest :

| Fichier | Test | Résultat |
|---|---|---|
| test_api.py | test_register — Inscription nouvel utilisateur | ✔ OK |
| test_api.py | test_login — Connexion avec identifiants valides | ✔ OK |
| test_api.py | test_login_invalid — Connexion avec mauvais mot de passe | ✔ OK (401) |
| test_api.py | test_get_profile — Récupérer profil avec token | ✔ OK |
| test_api.py | test_get_terrains — Liste des terrains (public) | ✔ OK |
| test_api.py | test_create_reservation — Créer une réservation | ✔ OK |
| test_api.py | test_conflict_detection — Conflit terrain/créneau identique | ✔ OK (409) |
| test_api.py | test_quota_limit — Dépassement quota 2/semaine | ✔ OK (429) |
| test_logic.py | test_total_cost_calculation — Coût = durée × tarif | ✔ OK |
| test_logic.py | test_time_overlap_detection — Détection chevauchement | ✔ OK |
| test_logic.py | test_weekly_quota_logic — Comptage 7 jours glissants | ✔ OK |
| test_logic.py | test_bcrypt_hash_compatible — Compatibilité hash app desktop | ✔ OK |

COMPÉTENCES MISES EN ŒUVRE

• Conception et développement d'une API RESTful sécurisée (authentification JWT, autorisation par rôle RBAC)
• Développement frontend React/TypeScript avec gestion d'état (AuthContext, hooks)
• Modélisation et gestion d'une base de données relationnelle (MCD Merise, MLD, contraintes d'intégrité)
• Mise en place d'une suite de tests automatisés pytest couvrant les endpoints et la logique métier
• Utilisation de Git/GitHub pour la gestion de version (branche develop)